

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHM

Experiment 8: Insertion Sort with Duplicate Elements

Aim

To implement Insertion Sort on an array containing duplicate elements and verify that it sorts correctly while keeping equal elements in their original order (stable).

Algorithm

1. Treat the first element as a sorted part.
2. Pick the next element as key.
3. Compare key with the elements in the sorted part, moving from right to left.
4. Shift every element that is **greater than** key one position to the right. Equal elements are not shifted, which keeps the sort stable.
5. Insert key into the gap.
6. Repeat steps 2 to 5 for all elements up to the end of the array.

Pseudo code

InsertionSort(A, n)

for i = 1 to n-1

 key = A[i]

 j = i - 1

 while j >= 0 and A[j] > key

 A[j+1] = A[j]

 j = j - 1

 A[j+1] = key

Source code (C)

```
#include <stdio.h>
```

```
void printArray(int a[], int n) {
```

```
    int i;
```

```
    for (i = 0; i < n; i++)
```

```

        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int a[] = {5, 3, 8, 3, 5, 1, 8, 3};
    int n = 8, i, j, key;

    printf("Original array: ");
    printArray(a, n);

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;

        printf("After pass %d : ", i);
        printArray(a, n);
    }

    printf("Sorted array : ");
    printArray(a, n);
    return 0;
}

```

Output

Original array: 5 3 8 3 5 1 8 3

After pass 1 : 3 5 8 3 5 1 8 3

After pass 2 : 3 5 8 3 5 1 8 3

After pass 3 : 3 3 5 8 5 1 8 3

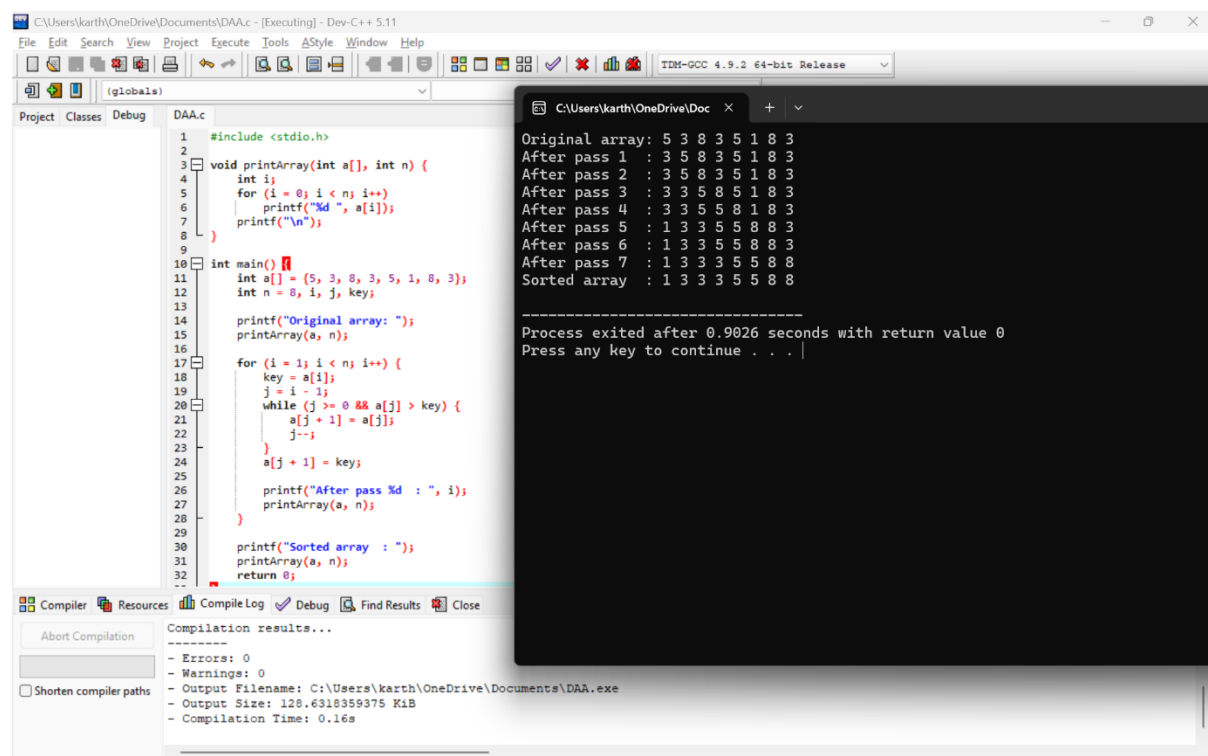
After pass 4 : 3 3 5 5 8 1 8 3

After pass 5 : 1 3 3 5 5 8 8 3

After pass 6 : 1 3 3 5 5 8 8 3

After pass 7 : 1 3 3 3 5 5 8 8

Sorted array : 1 3 3 3 5 5 8 8



The screenshot displays a C++ IDE with the source code for an Insertion Sort algorithm. The code defines a `printArray` function and a `main` function that initializes an array `a` with the values `{5, 3, 8, 3, 5, 1, 8, 3}` and `n = 8`. It then performs seven passes of the sorting algorithm, printing the array state after each pass, and finally prints the sorted array. The output window on the right shows the execution results, which match the text provided in the previous blocks. The compilation results at the bottom indicate that the program compiled successfully with no errors or warnings.

```
1 #include <stdio.h>
2
3 void printArray(int a[], int n) {
4     int i;
5     for (i = 0; i < n; i++)
6         printf("%d ", a[i]);
7     printf("\n");
8 }
9
10 int main()
11 {
12     int a[] = {5, 3, 8, 3, 5, 1, 8, 3};
13     int n = 8, i, j, key;
14
15     printf("Original array: ");
16     printArray(a, n);
17
18     for (i = 1; i < n; i++) {
19         key = a[i];
20         j = i - 1;
21         while (j >= 0 && a[j] > key) {
22             a[j + 1] = a[j];
23             j--;
24         }
25         a[j + 1] = key;
26
27         printf("After pass %d : ", i);
28         printArray(a, n);
29     }
30
31     printf("Sorted array : ");
32     printArray(a, n);
33     return 0;
34 }
```

Original array: 5 3 8 3 5 1 8 3
After pass 1 : 3 5 8 3 5 1 8 3
After pass 2 : 3 5 8 3 5 1 8 3
After pass 3 : 3 3 5 8 5 1 8 3
After pass 4 : 3 3 5 5 8 1 8 3
After pass 5 : 1 3 3 5 5 8 8 3
After pass 6 : 1 3 3 5 5 8 8 3
After pass 7 : 1 3 3 3 5 5 8 8
Sorted array : 1 3 3 3 5 5 8 8

Process exited after 0.9026 seconds with return value 0
Press any key to continue . . .

Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\karth\OneDrive\Documents\DAA.exe
- Output Size: 128.6318359375 KiB
- Compilation Time: 0.16s

Result

Insertion Sort sorted the array containing duplicate elements correctly. Because the shifting condition is `a[j] > key` (strictly greater), equal elements keep their relative order, so the algorithm is stable. The time complexity is $O(n^2)$ in the worst and average cases and $O(n)$ in the best case (already sorted).